

1. Call by Reference - Swap Function.cpp

This program demonstrates one of the most important concepts in C++: **Call by Reference** using **references (&)**

- **Function Signature:** void swap(int &num1, int &num2) — The ampersand (&) means these parameters are **references** to the original variables not copies.
- Inside the swap() function, we use a temporary variable tmp to exchange the values. Because num1 and num2 are references, changes made inside the function **directly modify** the original variables n1 and n2 in main()
- **Key Difference from Call by Value:** If we removed the &, the function would only swap copies, and the original variables would remain unchanged (10 20).
- **Output:** 20 10

2. class implementation

This is a **complete single-file Object-Oriented** implementation of a Time class.

- **Encapsulation:** Data members (hour, minute, second) are private, protecting them from direct external modification.
- **Constructor with Default Parameters:** Time(int h = 0, int m = 0, int s = 0) allows creating objects with or without arguments.
- **Data Validation:** The setTime() function uses the **ternary operator** to ensure values stay within valid ranges (24-hour format).
- **Constructor Delegation:** The constructor calls setTime() to reuse validation logic.
- **Demonstrates:** Constructor initialization, member functions, data hiding, and input sanitization.

3. constructor

This is a **basic version** of the Time class focused on constructors.

- It defines a **default constructor** (Time()) that runs automatically when an object is created.
- All member variables are initialized to 0.
- The display() function is a simple member function to output the time.

- **Purpose:** To teach the concept of **automatic initialization** of objects, which is a core benefit of using constructors in OOP.

This is typically the first step before introducing parameterized constructors.

4. hello.cpp

The legendary "**Hello World**" program — the traditional first program in almost every programming language.

- `#include <iostream>`: Includes the input/output stream library.
- `std::cout`: The standard output stream (console).
- `<<`: Stream insertion operator.
- `std::endl`: Outputs a newline and flushes the buffer.
- `return 0;`: Indicates successful program termination.

This program teaches basic syntax, program structure, and how to output text in C++.

5. overload

This program demonstrates **Function Overloading** — a form of **compile-time polymorphism**.

- Two functions have the **same name** (adder) but different **signatures** (number of parameters).
- The compiler decides which function to call based on the number and types of arguments.
- **Output:**

text

15

35

This feature allows more intuitive and readable code by using the same name for similar operations.

6. reference

This code compares and contrasts **Pointers** and **References** in C++.

- `int &ref = num;`: `ref` becomes another name for `num`.

- `int *ptr = #`: `ptr` stores the memory address of `num`.
- `int *&pref = ptr;`: A reference to a pointer.
- All four `cout` statements print 10, showing that references and properly dereferenced pointers access the same memory location.

Key Takeaway: References are safer and cleaner than pointers for most cases, but pointers offer more flexibility (e.g., nullability, re-assignment).

7. scan.cpp

This program shows how to perform **input** in C++ using `std::cin`.

- `std::cin >> num;` reads an integer.
- `std::cin >> str;` reads a word (stops at whitespace).
- It serves as a modern C++ replacement for C's `scanf()`.
- Note: For reading full lines with spaces, `std::getline()` would be better.

This is typically taught when transitioning from C to C++ input/output.

8. time structure

This is the **implementation file** for the `Time` class.

- It contains the actual **definitions** (bodies) of the member functions declared in `Time.h`.
- Uses **scope resolution operator** `Time::` to indicate these functions belong to the `Time` class.
- Part of the standard **multi-file project** structure in C++ (Header + Implementation + Main).